

Problema A

ESTIMATIVA

Arquivo fonte: `estimativa.{c|cpp|java|py}`

Autor: Emanuel Mineda Carneiro (Fatec São José dos Campos)

Um Scrum Master está preocupado com a carga de trabalho de sua equipe e precisa de sua ajuda para limitar a quantidade de User Stories a serem desenvolvidas em cada Sprint. Ele possui um Sprint Backlog priorizado e estimado. Além disso, conta também com a velocidade média da equipe. Sua tarefa consiste em desenvolver um programa capaz de, com essas informações, indicar quantas User Stories serão definidas como Meta da Sprint, levando em consideração que o custo total não deve exceder a velocidade média da equipe.

Exemplo:

Estimativa para as User Stories em ordem de prioridade: {3, 5, 8, 5, 1}
Velocidade média da equipe: 20
Quantidade de User Stories incluídas na Meta: 3 (as três primeiras, com estimativas 3, 5 e 8, respectivamente)
Isso ocorre porque se tentarmos incluir 4, o custo total passa a ser 21 ($3 + 5 + 8 + 5$), que excede a velocidade média (20)

Crie um programa capaz de indicar quantas User Stories incluir na meta, recebendo o Sprint Backlog priorizado e a velocidade média da equipe.

Entrada

A primeira linha contém o total de User Stories no Sprint Backlog. A segunda linha contém a velocidade média da equipe. A terceira linha apresenta as estimativas priorizadas de todas as User Stories do Sprint Backlog, separadas por espaço simples.

Saída

Imprimir, em linhas separadas, a quantidade de User Stories a incluir na meta e o custo total delas.

Exemplo de Entrada 1: 2 20 3 5 8 5 1	Exemplo de Saída 1: 3 16
Exemplo de Entrada 2: 6 25 3 5 8 5 8 3	Exemplo de Saída 2: 4 21

Problema B

BEMFORMADA

Arquivo fonte: *bemformada*.{c | cpp | java | py}

Autor: Fernando Masanori (Fatec São José dos Campos)

Em linguagens de programação e expressões matemáticas, o balanceamento de delimitadores é fundamental para a sintaxe correta. Sua tarefa é escrever um programa que verifique se uma sequência de chaves {}, colchetes [] e parênteses () está bem formada.

Uma sequência é considerada bem formada se:

- Todo símbolo de abertura é fechado por um símbolo correspondente do mesmo tipo.
- O fechamento ocorre na ordem inversa da abertura (LIFO — o último a abrir é o primeiro a fechar).
- Ao final do processamento, não restam símbolos de abertura aguardando fechamento.

Entrada

A primeira linha da entrada contém um inteiro **N**, que representa o número de casos de teste a serem processados. As **N** linhas seguintes contém, cada uma, uma string composta exclusivamente pelos caracteres (,), {, }, [e].

Saída

Para cada um dos **N** casos de teste, imprima uma única linha contendo o caractere **S** (se a expressão estiver bem formada) ou **N** (se não estiver).

Exemplo de Entrada 1:

2
{[()] }
{() }

Exemplo de Saída 1:

S
N

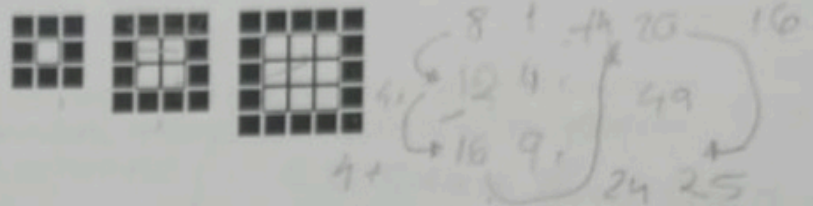
Problema C

MOSAICO DE AZULEJOS

Arquivo fonte: `mosaico.{c|cpp|java|py}`

Autor: Reinaldo Arakaki (Fatec São José dos Campos) adaptado

João é um azulejista que cria mosaicos quadrados. Ela é construída com azulejos quadrados pretos e brancos, todos do mesmo tamanho, sendo o primeiro formado por um azulejo branco cercado por azulejos pretos, o segundo por 4 azulejos brancos cercados por 12 azulejos pretos e assim, sucessivamente, como indica a figura. Porém ele precisa saber se com o número de azulejos pretos disponível quantos azulejos brancos serão utilizados, para criar o máximo de mosaicos possível!



Entrada

A entrada consiste em uma única linha contendo um inteiro $1 \leq N \leq 1000$, indicando, o número de azulejos pretos disponível.

Saída

Seu programa deverá imprimir uma única linha contendo um único valor correspondente ao número de azulejos brancos necessários, e caso não seja possível imprima 0.

Exemplo de Entrada 1: 80	Exemplo de Saída 1: 55
Exemplo de Entrada 2: 108	Exemplo de Saída 2: 91
Exemplo de Entrada 3: 5	Exemplo de Saída 3: 0
Exemplo de Entrada 3: 8	Exemplo de Saída 3: 1
Exemplo de Entrada 3: 10	Exemplo de Saída 3: 1

Problema D

O DILEMA DO VIAJANTE LITORÂNEO

Arquivo fonte: `viajante.{c|cpp|java|py}`

Autor: Fernando Masanori (Fatec São José dos Campos)

Em um mundo onde usamos aplicativos de navegação para otimizar tempo ou distância, um motorista precisa decidir qual o melhor caminho para chegar ao seu destino. Ele solicitou aos estudantes da FATEC um programa que calcule a rota ideal. Seja cruzando a Serra do Mar para chegar a Ubatuba ou navegando por grafos abstratos, o objetivo é o mesmo: **minimizar o custo total da viagem**. Sua missão é escrever um algoritmo que encontre a rota de menor peso entre um ponto de partida e um ponto de chegada, dado um mapa de conexões.

Entrada

A primeira linha da entrada contém dois inteiros, N e M , representando o número de cidades e o número de estradas (conexões), respectivamente. As M linhas seguintes contém a descrição de cada estrada no formato:

Origem Destino Peso

Onde Origem e Destino são strings (sem espaços) e Peso é um inteiro positivo.

A última linha da entrada contém duas strings: o nome da cidade de **Início** e o nome da cidade de **Destino**.

Saída

A saída deve conter duas linhas. A primeira linha deve exibir a sequência de cidades que formam o caminho de custo mínimo, separadas por espaço, no formato:
Caminho: Cidade1 Cidade2 ... CidadeN.

A segunda linha deve exibir o valor total do peso acumulado no formato: Custo Total: X.

Caso não haja caminho possível, imprima apenas Caminho inexistente.

Exemplo de Entrada 1:	Exemplo de Saída 1:
4 3 SJC Taubate 35 SJC Caraguatatuba 90 Taubate Ubatuba 70 SJC Ubatuba	Caminho: SJC Taubate Ubatuba Custo Total: 105

Dicas para o competidor:

- **Atenção aos espaços:** O BOCA é sensível a espaços extras no final das linhas. Use `print(f"Caminho: {' '.join(rota)}")`.
- **Eficiência:** Para garantir o "Accepted" (AC) dentro do limite de tempo, utilize uma fila de prioridade (heapq em Python).

Problema E

ALUNOS FALTANTES

Arquivo fonte: `aluno.{c|cpp|java|py}`

Autor: Reinaldo Arakaki (Fatec São José dos Campos)

O professor Alo Prado, cansado de anotar manualmente as presenças na chamada, pediu ao monitor da disciplina para desenvolver um programa que automatize o processo.

O programa lê o número total de alunos da turma e a lista de alunos presentes (identificados por números de 1 a N), exibindo os números dos alunos ausentes.

Entrada

- Primeira linha: Um inteiro N ($1 \leq N \leq 200$), o número de alunos na turma (numerados de 1 a N).
- Segunda linha: Uma lista de M inteiros ($1 \leq M \leq N$), não necessariamente em ordem, representando os alunos presentes. Os números são distintos e estão entre 1 e N.

Saída

Os números dos alunos ausentes (de 1 a N que não aparecem na lista de presentes), em ordem crescente. Se não houver faltas, imprimir OK.

Exemplo de Entrada 1: 5 2 3 1 5 4	Exemplo de Saída 1: OK
Exemplo de Entrada 2: 10 10 9 8 1 2 3 4 5 6 7	Exemplo de Saída 2: OK
Exemplo de Entrada 3: 5 1 3 5	Exemplo de Saída 3: 2 4

Problema F

OUVIDO ABSOLUTO

Arquivo fonte: `ouvido.{c|cpp|java|py}`

Autor: Wyllian(Formado-Fatec SJC)

Jhon é um engenheiro de software e músico amador que possui o chamado "ouvido absoluto". Ele consegue identificar se uma música é um plágio de outra apenas ouvindo os intervalos entre as notas. Na teoria musical, as notas são organizadas em um ciclo de 12 semitons (Dó, Dó#, Ré, Ré#, Mi, Fá, Fá#, Sol, Sol#, Lá, Lá#, Si).

No sistema computacional de Jhon, essas notas são representadas por números de 0 a 11. Uma melodia é considerada um PLÁGIO de outra se a sequência de intervalos for idêntica, independentemente do tom original.

Por exemplo, a sequência [0, 2] (Dó para Ré) é a mesma melodia que [7, 9] (Sol para Lá), pois em ambos os casos houve um salto de 2 semitons. Note que o sistema é cíclico: após a nota 11, a próxima nota é a 0.

Regras

- **Representação:** As notas nas fitas são números inteiros no intervalo 0,11.
- **Ciclo Tonal:** O cálculo de distância entre as notas deve considerar o módulo 12. Assim, o intervalo entre a nota 11 e a nota 0 é de +1 semitom.
- **Definição de Plágio:** Duas fitas são plágio se a diferença entre cada nota correspondente for constante em módulo 12. Ou seja, a Fita 2 deve ser uma transposição exata da Fita 1.
- **Tamanho:** Ambas as fitas fornecidas em um caso de teste sempre terão o mesmo tamanho N.

Entrada

A primeira linha da entrada contém um inteiro T, indicando o número de casos de teste. Para cada caso de teste, seguem três linhas:

- Um inteiro N ($2 \leq N \leq 10^5$), o número de notas em cada fita.
- N inteiros separados por espaço, representando as notas da Fita 1.
- N inteiros separados por espaço, representando as notas da Fita 2

Saída

Para cada caso de teste, imprima uma linha contendo a palavra **PLAGIO** ou **ORIGINAL**.

Exemplo de Entrada 1:	Exemplo de Saída 1:
1	PLAGIO
3	
1102	
346	

Exemplo de Entrada 2:	Exemplo de Saída 2:
1	ORIGINAL
2	
04	
03	

Explicação Exemplo 1:

Na Fita 1, os intervalos são +1 (de 11 para 0) e +2 (de 0 para 2).

Na Fita 2, os intervalos são +1 (de 3 para 4) e +2 (de 4 para 6).

Como a sequência de intervalos +1,+2 é idêntica, as melodias são as mesmas.

Explicação Exemplo 2:

A Fita 1 apresenta um intervalo de 4 semitons.

A Fita 2 apresenta um intervalo de 3 semitons. Como os intervalos são diferentes, a música é considerada original.

Problema G

MARATONA

fonte: maratona.{c|cpp|java|py}

Autor: Wyllian(Formado -Fatec São José dos Campos)

A comissão organizadora da Maratona está com problemas para processar os resultados da última corrida. Devido a uma falha no sistema de sensores, alguns corredores largaram antes do sinal oficial. De acordo com as regras da federação, qualquer atleta que realize a **largada antes do horário oficial** deve ser imediatamente desclassificado.

Sua tarefa é escrever um programa que receba o horário oficial da largada e os dados de desempenho de cada corredor, filtrando os desclassificados e determinando quem foi o grande vencedor.

O vencedor é definido como o corredor que, entre todos os que realizaram uma largada válida (no horário oficial ou após), cruzou a linha de chegada primeiro. Caso haja empate no horário de chegada entre dois ou mais corredores válidos, o critério de desempate será o menor ID.

Entrada

A entrada consiste em:

1. Uma linha contendo o Horário Oficial de Largada no formato HH MM SS.
2. Uma linha contendo um inteiro C ($1 \leq C \leq 10^4$), representando o número de corredores.
3. C linhas, cada uma contendo:
 - Um inteiro ID (Identificador do corredor).
 - O Horário de Largada Real do corredor (HH MM SS).
 - O Horário de Chegada do corredor (HH MM SS).

Saída

Imprima o **ID** do vencedor. Caso todos os corredores tenham sido desclassificados por queimar a largada, imprima a mensagem: SEM VENCEDOR.

Restrições

Os horários estão no formato de 24 horas.

O tempo total de prova de um corredor nunca excederá 24 horas.

Exemplo de Entrada 1: 08 00 00 3 101 07 59 59 09 00 00 202 08 00 05 09 00 05 303 08 00 10 09 00 15	Exemplo de Saída 1: 202
Exemplo de Entrada 2: 10 00 00 2 50 09 55 00 11 00 00 60 09 59 59 10 30 00	Exemplo de Saída 2: SEM VENCEDOR

Explicação Exemplo 1:

- O corredor **101** largou às 07:59:59 (antes das 08:00:00), logo está **desclassificado**.
- O corredor **202** levou exatamente 01:00:00 para completar a prova.
- O corredor **303** levou 01:00:05 para completar a prova.
O vencedor é o **202**.

Explicação Exemplo 2:

O corredor **50** largou 5 minutos antes do horário oficial.

O corredor **60** largou 1 segundo antes do horário oficial.

Como não restaram competidores válidos, a saída deve indicar que não houve vencedor.